

September 19, 2025

1 Neural Network

A neural network is inspired by the structure of the human brain. Its basic building block is a neuron, which takes inputs, performs computations, and produces an output. Each neuron has an associated activation function that determines whether it should “fire” based on its inputs.

Training a neural network requires to understand a few naming conventions:

1. **Architecture and Layers** A simple neural network consists of an input layer, a hidden layer and an output layer. For deep neural networks, the hidden layers should be more than one and possibly sometimes the number of input features are also larger.

Simple Neural Network x_1 , x_2 and x_3 are the input features and y_1 and y_2 are the outputs. Here, we can say, the model architecture is receiving 3 features and 2 outputs(2 classes) in a classification model.

2. Parameters In between each layers, there are some linear functions which consist of trainable parameters called weights and biases. Basically, the training of a neural network is updating these weights and biases to achieve the objective function of the specific problem.

Linear function with parameters Let's assume that x is an input feature from a data, the linear function is such a way that the weight w multiple with the input feature x and sum up with the bias b . In below graph, we explicitly assign $w = 1/2$ and $b = 1$ for easy understanding, and it shows a linear line representing that function. This example is only happening in one neuron or one feature. However in neural networks, there will be a lot of similar operations happening in parallel. In such cases, the vectorization and matrix operations are used for faster and more efficient computations.

3. Activation Function The activation function introduces non-linearity to the network. It decides whether a neuron should be active or not based on its weighted inputs. Common activation functions include sigmoid, tanh, and Rectified Linear Unit (ReLU).

Deep Neural Network Architecture image referenced from: [Industry 4.0 Interoperability, Analytics, Security, and Case Studies](#)

4. Feedforward In the feedforward neural network, the calculation is only forward pass without updating any parameters yet. The idea is first we calculate the linear function using the input features, then again from the outputs of the linear function, an activation function is introduced in

order to get the non-linear outputs. Then the activated outputs from the previous layer go to the next layer as the input of that layer. The process is generally repeated through all the layers until we get the final result y_i which is the predicted output. In feedforward, the predicted output may be some way different from the ground truth class.

5. Loss Function A loss function measures how far off the neural network's output is from the expected output(label or ground truth). It quantifies the network's performance and guides the learning process. Generally, in deep learning, minimizing the loss between the predicted output and the ground truth is the objective function of the model.

6. Optimizer An optimizer is an algorithm or a method used to adjust the parameters (weights and biases) of a neural network during training in order to minimize the loss function(objective function) or maximize the efficiency of the model. The optimizer helps in adjusting the learning rate, momentum and direction of the gradient calculation during training. Examples of optimizer:

1. Stochastic Gradient Descent(SGD)
2. Adam
3. RMSProp
4. Adagrad and so on...

7. Backpropagation Backpropagation is the process of updating the weights and biases in the network to minimize the loss function(objective function). It calculates the gradient of the loss with respect to the network's parameters and updates them using optimization algorithms like Gradient Descent.

Computational Graph

Derivation

Derivative Examples $f(x) = x^2 \gg \frac{\partial}{\partial x} f(x) = 2x$

$f(x) = x^3 \gg \frac{\partial}{\partial x} f(x) = 3x^2$

$f(x) = \log_e(x)$ or $f(x) = \ln(x) \gg \frac{\partial}{\partial x} f(x) = \frac{1}{x}$

$f(x) = e^x \gg \frac{\partial}{\partial x} f(x) = e^x$

$f(x) = 2x^2 + 3x + b \gg \frac{\partial}{\partial x} f(x) = 4x + 3$ (The derivative of a constant is always zero)

8. Training The network is trained using labeled data. During training, the input data is passed through the network, the output is compared to the expected output, and the loss is calculated. Backpropagation then adjusts the weights and biases to minimize the loss so that we will get the optimal weights and biases that fit with our problem.

Gradient Descent Gradient Descent is an optimization algorithm used in machine learning and deep learning to iteratively update the parameters of a model in order to minimize a loss function. The goal of the algorithm is to find the values of the model's parameters that result in the lowest possible value of the loss function, which in turn signifies the best fit of the model to the training data.

1. Initialization: The algorithm starts with an initial guess for the model's parameters.
2. Compute Loss: The current model's parameters are used to make predictions on the training data. The difference between these predictions and the actual target values (the loss) is calculated using a chosen loss function.
3. Compute Gradient: The gradient of the loss with respect to each parameter is computed. The gradient indicates the direction and magnitude of the steepest increase in the loss function. This step involves taking partial derivatives of the loss function with respect to each parameter.
4. Update Parameters: The parameters are updated by subtracting a fraction of the gradient from the current parameter values. This fraction is called the learning rate. The learning rate determines the step size in the parameter space and influences the speed and stability of convergence.
5. Repeat: Steps 2 to 4 are repeated for a certain number of iterations or until the change in the loss function becomes small (convergence criterion).

Gradient descent algorithm reference: Deep Learning Specialization by Andrew Ng from [Coursera](#).

9. Validation and Evaluation After training, the network's performance is evaluated using validation data to ensure it's not overfitting, underfitting, or the model is learning. Once satisfied, the network is tested on unseen data to measure its real-world performance. The testing process is sometimes called inferencing.

```
[1]: import numpy as np
      from sklearn.datasets import load_digits
      import matplotlib.pyplot as plt
```

Then, load the dataset. and see the data and shape of X(inputs) and y(labels).

```
[2]: # Load data
      data = load_digits()

      y_indices = data.target # load the targets(labels)
      X = np.matrix(data.data) # load the input images

      print('y shape:', y_indices.shape)
      print('X shape:', X.shape)

      print('y data:', y_indices[0:15])
      print('X data:', X[0:5])

      data_size = X.shape[0]
      x_area = X.shape[1]
```

```

y shape: (1797,)
X shape: (1797, 64)
y data: [0 1 2 3 4 5 6 7 8 9 0 1 2 3 4]
X data: [[ 0.  0.  5. 13.  9.  1.  0.  0.  0.  0. 13. 15. 10. 15.  5.  0.  0.
 3.
 15.  2.  0. 11.  8.  0.  0.  4. 12.  0.  0.  8.  8.  0.  0.  5.  8.  0.
  0.  9.  8.  0.  0.  4. 11.  0.  1. 12.  7.  0.  0.  2. 14.  5. 10. 12.
  0.  0.  0.  0.  6. 13. 10.  0.  0.  0.]
[ 0.  0.  0. 12. 13.  5.  0.  0.  0.  0.  0. 11. 16.  9.  0.  0.  0.  0.
 3. 15. 16.  6.  0.  0.  0.  7. 15. 16. 16.  2.  0.  0.  0.  0.  1. 16.
16.  3.  0.  0.  0.  0.  1. 16. 16.  6.  0.  0.  0.  0.  1. 16. 16.  6.
 0.  0.  0.  0.  0. 11. 16. 10.  0.  0.]
[ 0.  0.  0.  4. 15. 12.  0.  0.  0.  0.  3. 16. 15. 14.  0.  0.  0.  0.
 8. 13.  8. 16.  0.  0.  0.  0.  1.  6. 15. 11.  0.  0.  0.  1.  8. 13.
15.  1.  0.  0.  0.  9. 16. 16.  5.  0.  0.  0.  0.  3. 13. 16. 16. 11.
 5.  0.  0.  0.  0.  3. 11. 16.  9.  0.]
[ 0.  0.  7. 15. 13.  1.  0.  0.  0.  8. 13.  6. 15.  4.  0.  0.  0.  2.
 1. 13. 13.  0.  0.  0.  0.  0.  2. 15. 11.  1.  0.  0.  0.  0.  0.  1.
12. 12.  1.  0.  0.  0.  0.  0.  1. 10.  8.  0.  0.  0.  8.  4.  5. 14.
 9.  0.  0.  0.  7. 13. 13.  9.  0.  0.]
[ 0.  0.  0.  1. 11.  0.  0.  0.  0.  0.  0.  7.  8.  0.  0.  0.  0.  0.
 1. 13.  6.  2.  2.  0.  0.  0.  7. 15.  0.  9.  8.  0.  0.  5. 16. 10.
 0. 16.  6.  0.  0.  4. 15. 16. 13. 16.  1.  0.  0.  0.  0.  3. 15. 10.
 0.  0.  0.  0.  0.  2. 16.  4.  0.  0.]]

```

The shape of X is (1797, 64) which means we have 1797 samples of data in (1, 64) long vector. Since each sample represents 1 D vector and the actual size of an image is 8x8, we need to convert the size of input X into 8x8.

```

[3]: def convert_to_one_hot(y, onehot_size):
      y_vect = np.array(y)
      one_hot = np.zeros((y_vect.size, onehot_size), dtype=float)
      one_hot[np.arange(y_vect.size), y_vect] = 1
      return one_hot

```

```

[4]: y = convert_to_one_hot(y_indices, 10)

```

```

[5]: def normalize(X):
      mean = np.mean(X)
      std = np.std(X)
      # print(mean)
      # print(std)

      # YOUR CODE HERE
      X_vect=np.nan_to_num(X)
      if std==0:
          XX = (X_vect-mean)
      else:

```

```

XX = (X_vect-mean)/std

return XX

```

```
[6]: XX = normalize(X)
```

Exercise 6 Random split the train set to be 60% of data and the rest is the test set.

Hint: use `np.arange()` for set index number from 0 to `data_size`. Random index can do by using `random.shuffle()`. Please make sure that you are using normalized data to split.

```
[7]: percent_train = 0.6
data_size = len(data.data)    # assuming data is already normalized

# arange index number from 0 to data_size
idx = np.arange(data_size)

# random shuffle idx (1 line)
np.random.seed(7)
np.random.shuffle(idx)

# calculate number of training set
m_train = int(percent_train * data_size)

# split train_idx and test_idx
train_idx = idx[0:m_train]
test_idx = idx[m_train:]

# split to X_train and X_test
X_train = data.data[train_idx]
X_test = data.data[test_idx]

# split to y_train y_test and y_test_indices
# (assuming you have labels y with same length as data)
y_train = y[train_idx]
y_test = y[test_idx]
y_test_indices = test_idx

print("Train X:", X_train)
print("Test X:", X_test)
print("Train y:", y_train)
print("Test y:", y_test)
print("Test indices:", y_test_indices)
```

```

Train X: [[ 0.  0.  0. ... 15.  5.  0.]
 [ 0.  0.  2. ...  5.  0.  0.]
 [ 0.  0. 11. ...  0.  0.  0.]
 ...

```

```

[ 0.  0.  5. ...  0.  0.  0.]
[ 0.  4. 13. ...  7.  0.  0.]
[ 0.  0.  6. ...  0.  0.  0.]]
Test X: [[ 0.  0.  2. ...  8.  0.  0.]
[ 0.  0.  0. ...  9.  0.  0.]
[ 0.  0.  8. ...  6.  0.  0.]
...
[ 0.  0.  8. ... 16. 12.  5.]
[ 0.  2. 13. ... 12.  1.  0.]
[ 0.  1. 10. ...  4.  0.  0.]]
Train y: [[0. 0. 0. ... 0. 0. 0.]
[1. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 0. 0. ... 1. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 1. 0. 0.]]
Test y: [[0. 0. 0. ... 0. 0. 0.]
[0. 1. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 1. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
Test indices: [1683  466 1320 1173  277 1069 1017 1188  722 1741  786 1379 1429
1229
1170  906  563  388   22 1098  124 1599  773  880   50 1252 1304 1105
 703  482 1703   66 1537  733 1595  801 1380 1371  907   31  417  881
 632 1394 1578   64 1677   77  491 1266 1360  532   65 1513  296  591
 767  724  626 1318 1735 1441  251 1626 1493 1696  982 1584  706  487
 273 1345 1494  963  877 1647 1621  411  667  149  644 1435  218  239
1131  792  191  333 1190  962 1187 1739 1478 1446  582 1701 1286  839
 627  323 1469 1160  778 1076  939  437  119 1366 1395 1026 1303   76
1053  900  371 1568 1762  941  577  819 1313 1332 1547 1238  886  968
 785 1374 1438  301  689  812 1061  236 1115 1226  499 1483 1079  794
1593  990 1263  934  387  390  743  352 1217  235 1526  480  567 1140
1750 1664 1743  120 1543 1024 1086  926  515  223 1321  613  843   52
1651  485 1707  818  155  600 1433  248  761 1633 1130  542  793 1141
1302  628  150 1768  291  381  686 1224  897  854 1071 1781 1582 1654
 891 1785  615  154 1193 1531 1779  322 1381 1282   82  512 1361  929
1459   10  831   32 1353  413  580 1383 1698  614 1045 1014   89 1587
 601 1237  133  555 1031 1399  320 1583 1211 1552  634  552 1055 1392
 796 1489 1264 1577  174  766  245 1080 1551  145   90  682  310   93
1401  924 1511 1035  243 1111  214 1197  998  853  425 1575 1142 1143
 226 1776    0  401 1103   84  162  114  670 1174 1363  889  625 1748
 952  807  258  624  826  234  135  616 1749  836  842  959  592 1314
1291  343 1477 1408  293  364  178  659  996   14 1110  165 1678 1359
 59  709  676 1378  780  424  292 1065 1704  329  429 1012   79 1052

```

```

937 834 749 890 254 1549 1747 1407 1337 1108 136 1135 1218 852
1202 723 859 969 496 950 805 129 337 1780 608 1148 195 1250
1292 1764 909 584 103 1653 1668 719 884 776 849 894 227 1680
1325 1299 126 405 775 821 711 1687 1060 1163 303 267 545 212
896 1222 916 675 1289 814 1265 342 549 1410 1214 966 1230 295
695 1452 73 605 104 645 1404 1752 1040 439 1019 91 1112 1570
908 1474 102 238 1525 942 1270 938 1518 1181 5 1625 965 18
1430 121 661 1510 274 433 811 1300 1335 1669 1688 181 184 406
865 520 1003 380 1434 1393 1424 427 78 1009 1760 607 1597 1777
1631 1057 1440 1514 1091 1101 1129 1235 1580 958 559 846 690 1519
1729 1667 436 490 1782 1643 1387 69 808 1184 11 144 917 1541
1451 639 176 994 440 521 1767 1733 1004 1255 1234 143 753 519
1788 1322 822 428 612 1505 1032 156 539 1530 1460 1290 461 576
1362 255 979 1185 1006 1521 560 208 910 1311 1 1365 269 265
914 230 1763 1119 1758 864 1066 873 1738 398 1634 45 161 991
727 629 288 1613 1059 1744 575 850 1339 911 1254 1627 932 1100
1207 247 476 85 469 225 774 331 233 588 475 1284 599 421
182 988 1000 1356 1462 49 1675 1029 529 1258 486 1742 497 1646
106 443 1512 646 1036 726 190 282 29 169 1001 1204 100 197
495 647 987 44 710 373 678 566 1672 847 460 1437 1155 39
434 973 1239 1007 1579 1590 1498 456 260 35 827 309 276 636
654 622 844 393 107 349 524 784 47 203 1792 1655 21 402
1107 1154 1439 989 741 1310 1147 33 787 1540 923 1043 1723 1243
579 481 931 356 1685 1248 759 92 1175 1415 1341 1472 803 745
1546 279 1607 764 604 1774 1373 1398 400 1728 257 944 1096 1219
1507 1161 494 772 550 1486 201 312 290 1213 763 1136 949 183
1176 1349 1727 1068 1468 1030 250 823 1099 1536 940 391 1456 1092
1191 904 730 554 366 345 1608 535 1422 1209 1372 1639 919 211
1603 502 537 1220 175]

```

```

[8]: # test function - do not remove

assert X_train.shape[0] == int(percent_train * data_size), "training size is_
↳incorrect"
assert data_size - m_train == X_test.shape[0], "test size is incorrect"
assert train_idx[0] != 0 and train_idx[25] != 25, "training indices are not_
↳shuffled"
assert X_train.shape == (m_train, XX.shape[1]) and y_train.shape[0] == m_train
assert X_test.shape == (data_size - m_train, XX.shape[1]) and y_test.shape[0]_
↳== data_size - m_train and y_test_indices.shape[0] == data_size - m_train

```

1.1 2. Forward Propagation

The neural network we are going to implement looks very similar to the below diagram. However, our input size is 8x8 flattened into (1, 64) vector.

A few steps we will implement in forward propagation or feed forward network.

- a. Building activation functions

b. Initializing parameters

c. Feed forward layer

1.1.1 Building Activation Functions with Numpy

Referenced from [Activation functions](#).

Sigmoid Function Sigmoid function is known as the logistic function. The equation can be written as:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Before using `np.exp()`, you will use `math.exp()` to implement the sigmoid function. You will then see why `np.exp()` is preferable to `math.exp()`.

Example

```
[9]: import math

def math_sigmoid(x):
    '''
    Compute sigmoid of x
    '''
    if isinstance(x, (int, float)):
        # x is a scalar
        return 1 / (1 + math.exp(-x))
    elif isinstance(x, list):
        # x is a list of scalars
        return [1 / (1 + math.exp(-xi)) for xi in x]
    # x = np.array(x)
    z = 1 / (1 + math.exp(-x))
    # YOUR CODE HERE
    # raise NotImplementedError()
    return z
```

```
[10]: # test function - do not remove
print(math_sigmoid(5))

assert math_sigmoid(10) > 0.9999, "Calculate error"
assert math_sigmoid(-10) < 0.0001, "Calculate error"
assert math_sigmoid(0) == 0.5, "Calculate error"
```

0.9933071490757153

Actually, we rarely use the “math” library in deep learning because the inputs of the functions are real numbers. In deep learning we mostly use matrices and vectors. This is why numpy is more useful.


```
[11]: ### One reason why we use "numpy" instead of "math" in Deep Learning ###
x = [1, 2, 3]
math_sigmoid(x) # you will see this give an error when you run it, because x is
↳ a vector.
```

```
[11]: [0.7310585786300049, 0.8807970779778823, 0.9525741268224334]
```

In fact, if $x = (x_1, x_2, \dots, x_n)$ is a row vector then will apply the exponential function to every element of x . The output will thus be:

$$np.exp(x) = (e^{x_1}, e^{x_2}, \dots, e^{x_n})$$

```
[12]: # example of np.exp
x = np.array([1, 2, 3])
print(np.exp(x)) # result is (exp(1), exp(2), exp(3))

[ 2.71828183  7.3890561 20.08553692]
```

Exercise 7

```
[13]: # Grade cell - do not remove
# Use Numpy library to build your sigmoid function again

def Sigmoid(x):

    # YOUR CODE HERE
    x=np.array(x)
    output = 1/(1+np.exp(-x))
    # raise NotImplementedError()
    return output
```

```
[14]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Sigmoid(a)
print(y_hat)

assert y_hat.shape[0] == 5, "sigmoid output is incorrect"
assert np.round(y_hat[0],4) == 0.7109, "sigmoid output is incorrect"
assert np.round(y_hat[1],4) == 0.5498, "sigmoid output is incorrect"
assert np.round(y_hat[2],4) == 0.5250, "sigmoid output is incorrect"
assert np.round(y_hat[3],4) == 0.4256, "sigmoid output is incorrect"
assert np.round(y_hat[4],4) == 0.3318, "sigmoid output is incorrect"
```

```
[0.7109495  0.549834  0.52497919 0.42555748 0.33181223]
```

Expected Output: [0.7109495 0.549834 0.52497919 0.42555748 0.33181223]

ReLU (Rectified Linear Unit) A recent invention which stands for Rectified Linear Units. The formula is deceptively simple: $\max(0, z)$. Despite its name and appearance, it's not linear and provides the same benefits as Sigmoid (i.e. the ability to learn nonlinear functions), but with better performance.

$$\text{ReLU}(x) = \max(0, x)$$

Exercise 8

```
[15]: # Grade cell - do not remove
```

```
def ReLu(x):  
  
    return np.maximum(0, x)  
  
    # if np.isscalar(x):  
    #     return x if x > 0 else 0  
    # else:  
    #     output = []  
    #     for val in x:  
    #         if val < 0:  
    #             output.append(0)  
    #         else:  
    #             output.append(val)
```

```
[16]: # test function - do not remove
```

```
a = np.array([.9, 0.2, 0.1, -0.3, -0.7])  
  
y_hat = ReLu(a)  
print(y_hat)  
  
assert y_hat.shape[0] == 5, "ReLU output is incorrect"  
assert y_hat[3] > a[3] and y_hat[3] == 0, "ReLU output is incorrect"  
assert y_hat[4] > a[4] and y_hat[4] == 0, "ReLU output is incorrect"  
assert y_hat[0] == a[0], "ReLU output is incorrect"  
assert y_hat[1] == a[1], "ReLU output is incorrect"  
assert y_hat[2] == a[2], "ReLU output is incorrect"
```

```
[0.9 0.2 0.1 0.  0. ]
```

Expected Output: [0.9 0.2 0.1 0. 0.]

Tanh (Hyperbolic Tangent) Tanh squashes a real-valued number to the range $[-1, 1]$. It's non-linear. But unlike Sigmoid, its output is zero-centered. Therefore, in practice the tanh non-linearity is always preferred to the sigmoid nonlinearity.

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Exercise 9

```
[17]: # Grade cell - do not remove

def Tanh(x):
    output = (np.exp(x) - np.exp(-x)) / (np.exp(x) + np.exp(-x))
    # YOUR CODE HERE
    # raise NotImplementedError()
    return output

[18]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Tanh(a)
print(y_hat)

assert y_hat.shape[0] == 5, "Tanh output is incorrect"
assert np.round(y_hat[0],4) == 0.7163, "Tanh output is incorrect"
assert np.round(y_hat[1],4) == 0.1974, "Tanh output is incorrect"
assert np.round(y_hat[2],4) == 0.0997, "Tanh output is incorrect"
assert np.round(y_hat[3],4) == -0.2913, "Tanh output is incorrect"
assert np.round(y_hat[4],4) == -0.6044, "Tanh output is incorrect"

[ 0.71629787  0.19737532  0.09966799 -0.29131261 -0.60436778]
```

Expected Output: [0.71629787 0.19737532 0.09966799 -0.29131261 -0.60436778]

Softmax Softmax function calculates the probabilities distribution of the event over ‘i’ different events. In general way of saying, this function will calculate the probabilities of each target class over all possible target classes. Later the calculated probabilities will be helpful for determining the target class for the given inputs.

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

Exercise 10

```
[19]: # Grade cell - do not remove

def Softmax(x):
    """
    Calculates the softmax for each row of the input x.

    The code should work for a row vector and also for matrices of shape (m,n).
    """
```

```

# numerator = math.exp(x)
# YOUR CODE HERE

# Handle scalar input
if np.isscalar(x):
    return 1.0 # Softmax of a single value is always 1

# Shift input for numerical stability
x_shifted = x - np.max(x)
exp_x = np.exp(x_shifted)
output = exp_x / np.sum(exp_x)

return output

```

```

[20]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Softmax(a)
print(y_hat)

assert y_hat.shape[0] == 5, "Softmax output is incorrect"
assert np.round(y_hat[0],4) == 0.4083, "Softmax output is incorrect"
assert np.round(y_hat[1],4) == 0.2028, "Softmax output is incorrect"
assert np.round(y_hat[2],4) == 0.1835, "Softmax output is incorrect"
assert np.round(y_hat[3],4) == 0.1230, "Softmax output is incorrect"
assert np.round(y_hat[4],4) == 0.0824, "Softmax output is incorrect"

```

```
[0.4083291  0.20277023 0.18347409 0.12298636 0.08244022]
```

Expected Output: [0.4083291 0.20277023 0.18347409 0.12298636 0.08244022]

1.1.2 Initializing Parameters

Each layer contains weight vector w and bias b . We will create a set of random values with a normal distribution with mean zero and standard distribution 0.1. We create 3 layers as below.

```

[21]: h2 = 5
      h1 = 6

# Initialize the weights at each node at each layer. We do not have weights at
↳ the input layer thus it is an empty list.
W = [[], np.random.normal(0,0.1,[x_area,h1]),
      np.random.normal(0,0.1,[h1,h2]),
      np.random.normal(0,0.1,[h2,10])]
# Initialize the bias at each node at each layer. We do not have bias at the
↳ input layer thus it is an empty list.
B = [[], np.random.normal(0,0.1,[h1,1]),

```

```

np.random.normal(0,0.1,[h2,1]),
np.random.normal(0,0.1,[10,1])

# putting all the non-linear activation functions into a list.
# Layer 0 input layer has no activation
# Layer 1 activation is ReLu
# Layer 2 activation is Sigmoid
# The output layer activation is Softmax
act_funcs = [None, ReLu, Sigmoid, Softmax]

L = len(W)-1

```

1.1.3 Feed Forward Layer

For input $x^{(i)}$ at i-th layer, the forward propagation can be calculated by:

$$z^{(i)} = \mathbf{W}^T x^{(i)} + \mathbf{b}$$

$$\hat{y}^{(i)} = a^{(i)} = \text{act}(z^{(i)})$$

Exercise 11 Create forward_layer function which input self-define activation function

Note: If input act_func as None, the output is linear activation function

Hint: use * for multiplication

```

[22]: # Grade cell - do not remove

def forward_layer(w, b, X, act_func):

    X = np.asarray(X).reshape(-1)    # (n,)
    w = np.asarray(w)
    b = np.asarray(b)

    n = X.size

    # element-wise
    if w.ndim == 1 or (w.ndim == 2 and 1 in w.shape):
        w_vec = w.reshape(-1)
        z = w_vec * X
        if np.isscalar(b) or b.size == 1:
            z = z + b
        else:
            # assume length-n
            z = z + b.reshape(-1)
        z = z.reshape(-1, 1)          # (n,1)

    # multi-neuron: w (k,n)

```

```

else:
    k, n_w = w.shape
    #  $z[i,j] = w[j,i] * X[i] + b[j]$ 
    z = X.reshape(n, 1) * w.T #  $(n,1)*(n,k) \rightarrow (n,k)$ 
    if np.isscalar(b) or b.size == 1:
        z = z + b
    else: # assume length-k
        z = z + b.reshape(1, -1)

if act_func is None:
    y_hat = z
else:
    y_hat = act_func(z)

return z, y_hat

```

```

[23]: # test function - do not remove

X = np.array([[.9, 0.2, 0.1, -0.3, -0.7]]).T

w = np.array([[0.2, 0.1, 1, 3, 0.5]])
b = np.array([[1]])

z1, y_hat1 = forward_layer(w, b, X, None)
b = np.array([[0.5]])
z2, y_hat2 = forward_layer(w, 0.5, X, None)
print('Linear output of y_hat1', y_hat1, 'and', y_hat2)

assert y_hat1[2,0] == 1.1
assert np.round(y_hat2[3,0], 4) == -0.4

```

```

Linear output of y_hat1 [[1.18]
 [1.02]
 [1.1 ]
 [0.1 ]
 [0.65]] and [[ 0.68]
 [ 0.52]
 [ 0.6 ]
 [-0.4 ]
 [ 0.15]]

```

```

[24]: # test function - do not remove

X = np.array([.9, 0.2, 0.1, -0.3, -0.7])

w = np.array([0.2, 0.1, 1, 3, 0.5])

```

```

z1, y_hat1 = forward_layer(w, 1, X, ReLu)
z2, y_hat2 = forward_layer(w, 0.5, X, ReLu)
print('ReLu output of y_hat1', y_hat1, 'and', y_hat2)

assert y_hat1[3] > 0, "Forward layer is incorrect"
assert y_hat2[3] == 0, "Forward layer is incorrect"

```

ReLu output of y_hat1 [[1.18]
 [1.02]
 [1.1]
 [0.1]
 [0.65]] and [[0.68]
 [0.52]
 [0.6]
 [0.]
 [0.15]]

[25]: *# test function - do not remove*

```

X = np.array([.9, 0.2, 0.1, -0.3, -0.7])

w = np.array([0.2, 0.1, 1, 3, 0.5])

z1, y_hat1 = forward_layer(w, 1, X, Tanh)
z2, y_hat2 = forward_layer(w, 0.5, X, Tanh)
print('Tanh output of y_hat1', y_hat1, 'and', y_hat2)

assert y_hat1.shape[0] == 5

```

Tanh output of y_hat1 [[0.82745161]
 [0.76986654]
 [0.80049902]
 [0.09966799]
 [0.57166997]] and [[0.5915194]
 [0.47770001]
 [0.53704957]
 [-0.37994896]
 [0.14888503]]

[26]: *# test function - do not remove*

```

X = np.array([.9, 0.2, 0.1, -0.3, -0.7])

w = np.array([0.2, 0.1, 1, 3, 0.5])

z1, y_hat1 = forward_layer(w, 1, X, Sigmoid)
z2, y_hat2 = forward_layer(w, 0.5, X, Sigmoid)

```

```
print('Sigmoid output of y_hat1', y_hat1, 'and', y_hat2)
```

```
Sigmoid output of y_hat1 [[0.7649478 ]
 [0.7349726 ]
 [0.75026011]
 [0.52497919]
 [0.65701046]] and [[0.6637387 ]
 [0.62714777]
 [0.64565631]
 [0.40131234]
 [0.53742985]]
```

```
[27]: # test function - do not remove
```

```
X = np.array([[.9, 0.2, 0.1, -0.3, -0.7]]).T

w = np.array([[0.2, 0.1, 1, 3, 0.5], [0.3, 0.5, 0.1, -0.3, -0.5]])
b = np.array([-1,3])

z1, y_hat1 = forward_layer(w, b, X, Softmax)
print('Linear output of z1', z1)
print('Softmax output of y_hat1', y_hat1)

assert y_hat1.shape == (5, 2), "Forward layer is incorrect"
```

```
Linear output of z1 [[-0.82  3.27]
 [-0.98  3.1 ]
 [-0.9   3.01]
 [-1.9   3.09]
 [-1.35  3.35]]
Softmax output of y_hat1 [[0.00364271 0.21761522]
 [0.00310411 0.18359431]
 [0.00336265 0.16779256]
 [0.00123705 0.18176751]
 [0.00214412 0.23573976]]
```

Expected Output:

```
Linear output of z1 [[-0.82 3.27]
 [-0.98 3.1 ]
 [-0.9 3.01]
 [-1.9 3.09]
 [-1.35 3.35]]
Softmax output of y_hat1 [[0.00364271 0.21761522]
 [0.00310411 0.18359431]
 [0.00336265 0.16779256]
 [0.00123705 0.18176751]
 [0.00214412 0.23573976]]
```

Exercise 12 Create full of forward propagation

Calculate only z_layer and a_layer

```
[28]: # Grade cell - do not remove
def forward_one_step(X, W, B, act_funcs):
    import numpy as np

    # Force X to a column vector (n_in, 1) no matter how it's passed
    X = np.asarray(X).reshape(-1, 1)

    L = len(W) - 1
    a = [X]
    z = [[]]
    delta = [[]]
    dW = [[]]
    db = [[]]

    for l in range(1, L + 1):
        Wl = np.asarray(W[l])                # (n_in, n_out)
        Bl = np.asarray(B[l]).reshape(-1, 1)  # (n_out, 1)

        # Forward:  $z_l = W_l^T a_{l-1} + b_l \rightarrow (n_{out}, 1)$ 
        z_l = Wl.T @ a[l - 1] + Bl

        # Activation:  $a_l = f_l(z_l)$  or identity
        a_l = z_l if (act_funcs[l] is None) else act_funcs[l](z_l)

        z.append(z_l)
        a.append(a_l)

        # Placeholders with correct shapes for backprop
        delta.append(np.zeros_like(a_l))  # (n_out, 1)
        dW.append(np.zeros_like(Wl))      # (n_in, n_out)
        db.append(np.zeros_like(Bl))      # (n_out, 1)

    return a, z, delta, dW, db
```

```
[29]: # test function - do not remove

x_this = X_train[0,:].T

a, z, delta, dW, db = forward_one_step(x_this, W, B, act_funcs)
print('size of a', len(a), 'a[3] =', a[3])
print('size of z', len(z), 'z[3] =', z[3])

assert len(a) == len(z) and len(a) == 4
assert a[0].shape == (64,1)
assert a[1].shape == (6,1)
```

```
assert a[2].shape == (5,1)
assert a[3].shape == (10,1)
```

```
size of a 4 a[3] = [[0.07747197]
 [0.10080759]
 [0.08890507]
 [0.06853561]
 [0.09798043]
 [0.12313608]
 [0.09477845]
 [0.11462841]
 [0.13331274]
 [0.10044363]]
size of z 4 z[3] = [[-0.18929484]
 [ 0.07400262]
 [-0.05164182]
 [-0.31185751]
 [ 0.04555675]
 [ 0.27407907]
 [ 0.01233106]
 [ 0.2024846 ]
 [ 0.35348673]
 [ 0.07038566]]
```

Expected Output (The output may not the same):

```
size of a 4 a[3] = [[0.09043188]
 [0.11294963]
 [0.06708026]
 [0.11812721]
 [0.11817167]
 [0.10483097]
 [0.09818055]
 [0.08948474]
 [0.10362842]
 [0.09711466]]
size of z 4 z[3] = [[-0.05075633]
 [ 0.17158877]
 [-0.34946347]
 [ 0.21640886]
 [ 0.2167852 ]
 [ 0.096996 ]
 [ 0.03145495]
 [-0.06128506]
 [ 0.0854584 ]
 [ 0.02053917]]
```

1.2 3. Loss function

For softmax loss function, it is cross entropy loss. You can calculate as

$$\mathcal{L} = - \sum_{i=0}^n (y_i * \log \hat{y}_i)$$

Exercise 13 Create loss function for multi classification (cross entropy loss)

```
[30]: # Grade cell - do not remove
```

```
def loss(y, yhat):
    # y=np.array(y)
    # yhat=np.array(yhat)
    epsilon = 1e-15
    yhat_safe = np.clip(yhat, epsilon, 1 - epsilon)

    l = -np.sum(y * np.log(yhat_safe))

    # YOUR CODE HERE
    # raise NotImplementedError()
    return l
```

```
[31]: # test function - do not remove
```

```
y_hat = np.array([0.4083291, 0.20277023, 0.18347409, 0.12298636, 0.08244022])
y = np.array([0, 1, 0, 0, 0])

l = loss(y, y_hat)
print(l)

assert np.round(l, 4) == 1.5957, "Loss function incorrect"
```

```
1.5956818129123256
```

Expected Output: 1.5956818129123256

1.3 4. Back propagation

Back propagation can be calculated as

$$\frac{\partial \mathcal{L}}{\partial z^{[l-1]}} = [W^{[l]}]^T \cdot \frac{\partial \mathcal{L}}{\partial z^{[l]}} * g^{[l-1]'}(z^{[l-1]})$$

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[l]}} \cdot [a^{[l-1]}]^T$$

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

When $g^{[l-1]'}$ is derivative activation function.

Thus first of all, we need to calculate derivative of the activation functions that we use.

The Linear_derivative (dl) function is

$$dl(x) = [1]$$

The ReLu_derivative ($dReLU$) function is

$$dReLU(x) = [1 \text{ when } x > 0, \text{ otherwise } 0]$$

The Tanh_derivative ($dTanh$) function is

$$dTanh(x) = 1 - \tanh^2(x)$$

The Sigmoid_derivative (ds) function is

$$ds(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$$

Exercise 14 Write the derivative functions as above

```
[32]: # Grade cell - do not remove
```

```
def Linear_derivative(x):
    output = np.ones_like(x)
    # YOUR CODE HERE

    return output
```

```
[33]: # test function - do not remove
```

```
a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Linear_derivative(a)
print(y_hat)

assert y_hat.shape[0] == 5, "Linear_derivative output is incorrect"
assert y_hat[0] == 1, "Linear_derivative output is incorrect"
assert y_hat[1] == 1, "Linear_derivative output is incorrect"
assert y_hat[2] == 1, "Linear_derivative output is incorrect"
assert y_hat[3] == 1, "Linear_derivative output is incorrect"
assert y_hat[4] == 1, "Linear_derivative output is incorrect"
```

```
[1. 1. 1. 1. 1.]
```

```
[34]: # Grade cell - do not remove
```

```
def ReLu_derivative(x):
    output = np.where(x > 0, 1, 0)
    # YOUR CODE HERE
```

```
return output
```

```
[35]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = ReLu_derivative(a)
print(y_hat)

assert y_hat.shape[0] == 5, "ReLu_derivative output is incorrect"
assert y_hat[0] == 1, "ReLu_derivative output is incorrect"
assert y_hat[1] == 1, "ReLu_derivative output is incorrect"
assert y_hat[2] == 1, "ReLu_derivative output is incorrect"
assert y_hat[3] == 0, "ReLu_derivative output is incorrect"
assert y_hat[4] == 0, "ReLu_derivative output is incorrect"
```

```
[1 1 1 0 0]
```

```
[36]: # Grade cell - do not remove

def Tanh_derivative(x):
    output = 1 - np.tanh(x) ** 2
    # YOUR CODE HERE

    return output
```

```
[37]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Tanh_derivative(a)
print(y_hat)

assert y_hat.shape[0] == 5, "Tanh_derivative output is incorrect"
assert np.round(y_hat[0],4) == 0.4869, "Tanh_derivative output is incorrect"
assert np.round(y_hat[1],4) == 0.9610, "Tanh_derivative output is incorrect"
assert np.round(y_hat[2],4) == 0.9901, "Tanh_derivative output is incorrect"
assert np.round(y_hat[3],4) == 0.9151, "Tanh_derivative output is incorrect"
assert np.round(y_hat[4],4) == 0.6347, "Tanh_derivative output is incorrect"
```

```
[0.48691736 0.96104298 0.99006629 0.91513696 0.63473959]
```

```
[38]: # Grade cell - do not remove

def Sigmoid_derivative(x):
    output = Sigmoid(x)*(1-Sigmoid(x))
    # YOUR CODE HERE
```

```
return output
```

```
[39]: # test function - do not remove

a = np.array([.9, 0.2, 0.1, -0.3, -0.7])

y_hat = Sigmoid_derivative(a)
print(y_hat)

assert y_hat.shape[0] == 5, "Sigmoid_derivative output is incorrect"
assert np.round(y_hat[0],4) == 0.2055, "Sigmoid_derivative output is incorrect"
assert np.round(y_hat[1],4) == 0.2475, "Sigmoid_derivative output is incorrect"
assert np.round(y_hat[2],4) == 0.2494, "Sigmoid_derivative output is incorrect"
assert np.round(y_hat[3],4) == 0.2445, "Sigmoid_derivative output is incorrect"
assert np.round(y_hat[4],4) == 0.2217, "Sigmoid_derivative output is incorrect"
```

```
[0.20550031 0.24751657 0.24937604 0.24445831 0.22171287]
```

Exercise 15 Create back propagation function.

```
[40]: # Grade cell - do not remove

def back_propagation(y, a, z, W, dW, db, act_der):
    '''
    Backprop step. Note that derivative of multinomial cross entropy
    loss is the same as that of binary cross entropy loss. See
    https://levelup.gitconnected.com/
    ↪killer-combo-softmax-and-cross-entropy-5907442f60ba
    for a nice derivation.
    '''
    L = len(W)-1
    y = np.asarray(y).reshape(-1, 1)
    delta = [None] * (L + 1)

    delta[L] = a[L] - y

    for l in range(L,0,-1):
        # db = delta(l)
        # Bias and weight gradients
        db[l] = delta[l] # (n_out, 1)
        dW[l] = a[l - 1] @ delta[l].T # (n_in, n_out) ← matches W[l]

    if l > 1:
        # activation derivative for layer l-1
```

```

        gprime_fn = act_der[1 - 1] if act_der[1 - 1] is not None else Linear_derivative
        # Backprop to previous layer: USE W[l], NOT W[l].T
        delta[1 - 1] = (W[1] @ delta[1]) * gprime_fn(z[1 - 1]) # (n_{l-1}, 1)
        1)

        # YOUR CODE HERE

    return dW, db

```

Create activation derivative variable

```
[41]: act_der = [None, ReLu_derivative, Sigmoid_derivative, Softmax]
```

```
[42]: # test function - do not remove

x_this = X_train[0,:].T
y_this = y_train[0,:]

a, z, delta, dW, db = forward_one_step(x_this, W, B, act_funcs)
dW, db = back_propagation(y_this, a, z, W, dW, db, act_der)

lenW = [0, 64, 6, 5]
for i in range(4):
    assert len(dW[i]) == lenW[i]

print("dW", dW)
print("db", db)

```

```

dW [[], array([[ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.01526055,  0.          ,  0.00216446, -0.01797932,
                0.02727025],
               [ 0.          ,  0.01983871,  0.          ,  0.0028138 , -0.02337312,
                0.03545132],
               [ 0.          ,  0.00152605,  0.          ,  0.00021645, -0.00179793,
                0.00272702],
               [ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                0.          ],
               [ 0.          ,  0.00152605,  0.          ,  0.00021645, -0.00179793,
                0.          ]])

```

```

0.00272702],
[ 0.          , 0.0167866 , 0.          , 0.00238091, -0.01977725,
 0.02999727],
[ 0.          , 0.01831266, 0.          , 0.00259735, -0.02157519,
 0.03272429],
[ 0.          , 0.01068238, 0.          , 0.00151512, -0.01258553,
 0.01908917],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.00305211, 0.          , 0.00043289, -0.00359586,
 0.00545405],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
 0.04363239],
[ 0.          , 0.01831266, 0.          , 0.00259735, -0.02157519,
 0.03272429],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.00610422, 0.          , 0.00086578, -0.00719173,
 0.0109081 ],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
 0.04363239],
[ 0.          , 0.0167866 , 0.          , 0.00238091, -0.01977725,
 0.02999727],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
 0.          ],
[ 0.          , 0.00610422, 0.          , 0.00086578, -0.00719173,

```



```

0.0109081 ],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
  0.04363239],
[ 0.          , 0.02289082, 0.          , 0.00324669, -0.02696898,
  0.04090537],
[ 0.          , 0.01220844, 0.          , 0.00173157, -0.01438346,
  0.0218162 ],
[ 0.          , 0.00610422, 0.          , 0.00086578, -0.00719173,
  0.0109081 ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.00610422, 0.          , 0.00086578, -0.00719173,
  0.0109081 ],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
  0.04363239],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
  0.04363239],
[ 0.          , 0.01983871, 0.          , 0.0028138 , -0.02337312,
  0.03545132],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
  0.04363239],
[ 0.          , 0.00915633, 0.          , 0.00129868, -0.01078759,
  0.01636215],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.01068238, 0.          , 0.00151512, -0.01258553,
  0.01908917],
[ 0.          , 0.02441688, 0.          , 0.00346314, -0.02876691,
  0.04363239],
[ 0.          , 0.01068238, 0.          , 0.00151512, -0.01258553,
  0.01908917],
[ 0.          , 0.01983871, 0.          , 0.0028138 , -0.02337312,
  0.03545132],
[ 0.          , 0.02136477, 0.          , 0.00303025, -0.02517105,
  0.03817834],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],
[ 0.          , 0.          , 0.          , 0.          , 0.          ,
  0.          ],

```

```

0.      ],
[ 0.      , 0.      , 0.      , 0.      , 0.      ,
0.      ],
[ 0.      , 0.01068238, 0.      , 0.00151512, -0.01258553,
0.01908917],
[ 0.      , 0.02289082, 0.      , 0.00324669, -0.02696898,
0.04090537],
[ 0.      , 0.02289082, 0.      , 0.00324669, -0.02696898,
0.04090537],
[ 0.      , 0.00763027, 0.      , 0.00108223, -0.00898966,
0.01363512],
[ 0.      , 0.      , 0.      , 0.      , 0.      ,
0.      ]]), array([[ 0.      , 0.      , 0.      , 0.
, 0.      ],
[ 0.04252687, 0.00657922, 0.00840036, -0.01442646, -0.00910979],
[ 0.      , 0.      , 0.      , 0.      , 0.      ],
[ 0.08249452, 0.01276251, 0.01629519, -0.02798475, -0.01767136],
[ 0.02590819, 0.00400819, 0.00511766, -0.00878888, -0.00554986],
[ 0.09682883, 0.01498013, 0.01912666, -0.0328474 , -0.02074195]]),
array([[ 0.05359045, 0.06973262, 0.06149918, 0.04740881, 0.06777696,
0.08517813, -0.62617776, 0.07929303, 0.09221773, 0.06948086],
[ 0.05635103, 0.07332473, 0.06466716, 0.04985096, 0.07126833,
0.08956587, -0.65843379, 0.08337762, 0.0969681 , 0.07306  ],
[ 0.03957724, 0.05149845, 0.04541794, 0.03501203, 0.05005417,
0.06290516, -0.46244044, 0.05855894, 0.06810399, 0.05131252],
[ 0.0599787 , 0.07804511, 0.0688302 , 0.05306019, 0.07585633,
0.0953318 , -0.70082141, 0.08874517, 0.10321056, 0.07776334],
[ 0.00642421, 0.00835927, 0.00737228, 0.00568318, 0.00812484,
0.01021082, -0.07506374, 0.00950534, 0.0110547 , 0.00832909]]])
db [[], array([[ 0.      ],
[ 0.00152605],
[-0.      ],
[ 0.00021645],
[-0.00179793],
[ 0.00272702]]), array([[ 0.01778974],
[ 0.0027522 ],
[ 0.00351402],
[-0.00603484],
[-0.00381078]]), array([[ 0.07747197],
[ 0.10080759],
[ 0.08890507],
[ 0.06853561],
[ 0.09798043],
[ 0.12313608],
[-0.90522155],
[ 0.11462841],
[ 0.13331274],
[ 0.10044363]]])

```

1.4 5. Parameters Update

In the training, to improve accuracy, you need to update weight/bias while training. Weight and bias update equations are

$$W_{new}^{(i)} = W_{old}^{(i)} - \alpha * \delta W$$

$$B_{new}^{(i)} = B_{old}^{(i)} - \alpha * \delta B$$

When α is learning rate. and i is the layer number of network

Exercise 16 Create update_step function

```
[43]: # Grade cell - do not remove

def update_step(W, B, dW, db, alpha):
    L = len(W)-1
    for l in range(1,L+1):
        # W[l] = None
        # B[l] = None
        # YOUR CODE HERE
        W[l] = np.asarray(W[l]) - alpha * np.asarray(dW[l])
        B[l] = np.asarray(B[l]) - alpha * np.asarray(db[l])

    return W, B

[44]: # test function - do not remove

x_this = X_train[0,:].T
y_this = y_train[0,:]

alpha = 0.1

a, z, delta, dW, db = forward_one_step(x_this, W, B, act_funcs)
dW, db = back_propagation(y_this, a, z, W, dW, db, act_der)

W_new, B_new = update_step(W, B, dW, db, alpha)
W_new_2, B_new_2 = update_step(W_new, B_new, dW, db, alpha)

result_w = np.array_equal(W, W_new)
result_w2 = np.array_equal(W_new, W_new_2)
assert W[2].shape == W_new[2].shape and W[1].shape == W_new_2[1].shape, "W_new_2
↪shape must be the same"
assert not result_w and not result_w2, "Weight must be updated"
```

```

result_b = np.array_equal(B, B_new)
result_b2 = np.array_equal(B_new, B_new_2)
assert B[3].shape == B_new[3].shape and B[1].shape == B_new_2[1].shape, "b_new_2
↳shape must be the same"
assert not result_b and not result_b2, "Bias must be updated"

```

1.5 Put it together

Exercise 17 Create training code using the functions above

```

[45]: # Grade cell - do not remove

import warnings

# Silence the specific RuntimeWarning and NumPy floating warnings
warnings.filterwarnings("ignore", message="overflow encountered in exp")
warnings.filterwarnings("ignore", message="invalid value encountered in divide")
np.seterr(over='ignore', invalid='ignore')

cost_arr = []

alpha = 0.0001          # smaller LR to avoid large logits that trigger exp
↳overflows
max_iter = 100
for iter in range(0, max_iter):
    loss_this_iter = 0.0
    order = np.random.permutation(m_train)

    for i in range(0, m_train):
        # Grab the pattern order[i]
        x_this = X_train[order[i], :].T
        y_this = y_train[order[i], :]

        # Feed forward step
        a, z, delta, dW, db = forward_one_step(x_this, W, B, act_funcs)

        # If forward produced non-finite activations, skip this pattern
        if not np.all(np.isfinite(a[-1])):
            continue

        # calculate loss for each epoch (use y_this, not y)
        a_final = a[-1] # last layer activation
        loss_this_pattern = loss(y_this, a_final)
        if not np.isfinite(loss_this_pattern):
            continue
        loss_this_iter += float(loss_this_pattern)

```

```

    # back propagation
    dW, db = back_propagation(y_this, a, z, W, dW, db, act_der)

    # Inline gradient clipping (prevents massive updates -> huge z -> exp_
    ↪ overflow)
    for l in range(1, len(dW)):
        if isinstance(dW[l], np.ndarray):
            np.clip(dW[l], -5.0, 5.0, out=dW[l])
        if isinstance(db[l], np.ndarray):
            np.clip(db[l], -5.0, 5.0, out=db[l])

    # Skip update if any gradient is non-finite
    grads_ok = True
    for g in list(dW[1:]) + list(db[1:]):
        if isinstance(g, np.ndarray) and not np.all(np.isfinite(g)):
            grads_ok = False
            break
    if not grads_ok:
        continue

    # update weight, bias
    W, B = update_step(W, B, dW, db, alpha)

    cost_arr.append(loss_this_iter)

```

```

[46]: # test function - do not remove

for i in range(max_iter):
    print('Epoch %d train loss %f' % (i + 1, cost_arr[i]))
assert len(cost_arr) == max_iter

```

```

Epoch 1 train loss 25134.495419
Epoch 2 train loss 25110.299744
Epoch 3 train loss 25092.106105
Epoch 4 train loss 25071.089464
Epoch 5 train loss 25055.974887
Epoch 6 train loss 25041.004806
Epoch 7 train loss 25026.841784
Epoch 8 train loss 25013.521476
Epoch 9 train loss 25001.598240
Epoch 10 train loss 24990.928689
Epoch 11 train loss 24980.821093
Epoch 12 train loss 24971.199558
Epoch 13 train loss 24963.221709
Epoch 14 train loss 24955.387926
Epoch 15 train loss 24947.414176
Epoch 16 train loss 24941.474265
Epoch 17 train loss 24934.712626

```

Epoch 18 train loss 24928.827634
Epoch 19 train loss 24923.454894
Epoch 20 train loss 24918.321420
Epoch 21 train loss 24914.574891
Epoch 22 train loss 24910.248416
Epoch 23 train loss 24906.064955
Epoch 24 train loss 24902.659445
Epoch 25 train loss 24898.893684
Epoch 26 train loss 24895.887156
Epoch 27 train loss 24893.424088
Epoch 28 train loss 24890.925159
Epoch 29 train loss 24888.851232
Epoch 30 train loss 24886.350279
Epoch 31 train loss 24884.457353
Epoch 32 train loss 24883.231548
Epoch 33 train loss 24881.623191
Epoch 34 train loss 24880.050411
Epoch 35 train loss 24878.954160
Epoch 36 train loss 24878.562258
Epoch 37 train loss 24877.513211
Epoch 38 train loss 24877.024710
Epoch 39 train loss 24877.103661
Epoch 40 train loss 24876.928039
Epoch 41 train loss 24876.636237
Epoch 42 train loss 24876.966830
Epoch 43 train loss 24876.967672
Epoch 44 train loss 24877.779870
Epoch 45 train loss 24879.079147
Epoch 46 train loss 24879.627987
Epoch 47 train loss 24880.493395
Epoch 48 train loss 24882.224341
Epoch 49 train loss 24883.928124
Epoch 50 train loss 24885.406564
Epoch 51 train loss 24887.786886
Epoch 52 train loss 24890.159138
Epoch 53 train loss 24892.803049
Epoch 54 train loss 24896.047422
Epoch 55 train loss 24899.865661
Epoch 56 train loss 24903.459549
Epoch 57 train loss 24907.308629
Epoch 58 train loss 24911.858911
Epoch 59 train loss 24916.388743
Epoch 60 train loss 24921.604553
Epoch 61 train loss 24927.046913
Epoch 62 train loss 24933.605077
Epoch 63 train loss 24940.182376
Epoch 64 train loss 24946.868988
Epoch 65 train loss 24954.092782

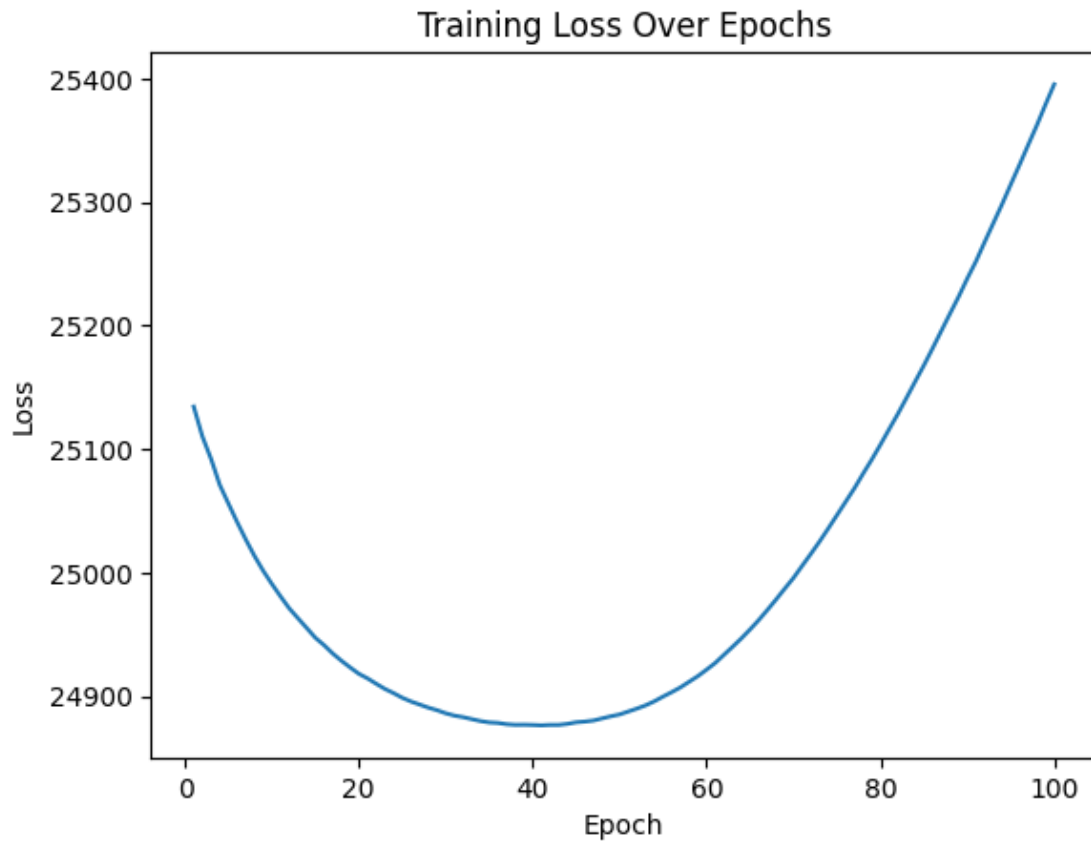
```
Epoch 66 train loss 24961.700445
Epoch 67 train loss 24969.907351
Epoch 68 train loss 24978.395460
Epoch 69 train loss 24987.171578
Epoch 70 train loss 24995.822472
Epoch 71 train loss 25005.628912
Epoch 72 train loss 25015.262037
Epoch 73 train loss 25025.282187
Epoch 74 train loss 25035.782971
Epoch 75 train loss 25046.550567
Epoch 76 train loss 25057.387370
Epoch 77 train loss 25068.203616
Epoch 78 train loss 25080.049049
Epoch 79 train loss 25091.459201
Epoch 80 train loss 25103.560282
Epoch 81 train loss 25116.038581
Epoch 82 train loss 25128.355914
Epoch 83 train loss 25141.408310
Epoch 84 train loss 25154.617953
Epoch 85 train loss 25167.718817
Epoch 86 train loss 25181.459071
Epoch 87 train loss 25195.512387
Epoch 88 train loss 25209.512531
Epoch 89 train loss 25223.270610
Epoch 90 train loss 25238.068760
Epoch 91 train loss 25251.949794
Epoch 92 train loss 25267.614104
Epoch 93 train loss 25282.867131
Epoch 94 train loss 25298.148622
Epoch 95 train loss 25314.140447
Epoch 96 train loss 25329.865748
Epoch 97 train loss 25346.007006
Epoch 98 train loss 25361.937745
Epoch 99 train loss 25378.848504
Epoch 100 train loss 25395.194629
```

1.5.1 Take Home Exercise

1. Plot the loss value into graph using pyplot
2. Create Prediction function to predict the test set which we have separated from above. Calculate the accuracy of prediction.

```
[47]: # YOUR CODE HERE
plt.figure()
plt.plot(range(1, len(cost_arr) + 1), cost_arr)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Over Epochs")
```

```
plt.show()
```



```
[48]: # print("y_pred shape:", y_pred.shape)
      # print("y_test shape:", np.asarray(y_test).shape)
      # print(f"Test accuracy: {accuracy:.4f}")
```

```
[49]: def predict_on_testset(X, W, B, act_funcs, y_true=None, threshold=0.5):
      """
      Predict on X using trained W, B, act_funcs.

      Returns:
      y_pred : (m,) integer class labels
      accuracy : float in [0,1] if y_true is provided, else None

      Notes:
      - Binary (n_out=1): threshold at 0.5
      - Multiclass (n_out>1): argmax
      - y_true can be (m,), (m,1), or one-hot (m,n_classes)
      """
      m = X.shape[0]
```



```

# Probe output size
a0, _, _, _, _ = forward_one_step(X[0, :].reshape(-1, 1), W, B, act_funcs)
n_out = a0[-1].reshape(-1, 1).shape[0]

# Predict labels
y_pred = np.empty(m, dtype=int)
for i in range(m):
    a, _, _, _, _ = forward_one_step(X[i, :].reshape(-1, 1), W, B,
    ↪ act_funcs)
    y_hat = a[-1].reshape(-1) # final activation
    if n_out == 1:
        y_pred[i] = int(y_hat[0] >= threshold)
    else:
        y_pred[i] = int(np.argmax(y_hat))

# Optional accuracy
accuracy = None
if y_true is not None:
    yt = np.asarray(y_true)
    if yt.ndim == 2:
        if yt.shape[1] == 1:
            y_true_labels = yt[:, 0].astype(int) # binary column
        else:
            y_true_labels = np.argmax(yt, axis=1).astype(int) # one-hot ->
    ↪ index
    else:
        y_true_labels = yt.reshape(-1).astype(int)

    if y_true_labels.shape[0] != y_pred.shape[0]:
        raise ValueError(
            f"Sample count mismatch: y_pred={y_pred.shape[0]} vs
    ↪ y_true={y_true_labels.shape[0]}"
        )
    accuracy = float(np.mean(y_pred == y_true_labels))

return y_pred, accuracy

```

```

[50]: y_pred, acc = predict_on_testset(X_test, W, B, act_funcs, y_true=y_test)
print(f"Test accuracy: {acc:.4f}")

```

Test accuracy: 0.4145

[]: